



AEGISEPC

TECHNICAL SPECIFICATION & WHITEPAPER

Next-Generation Engineering Quality Compliance & RFI Intelligence



An implementation of structural clause-tree retrieval and multi-agent compliance pipelines to eliminate procurement deviations and schedule bottlenecks in capital infrastructure projects.

Project Title: AegisEPC

Submission Stream: ET AI
Hackathon 2026

Core Engines: Gemini LLM /
Python FastAPI

Code Repository: [AI-Data-Centre-EPC-Platform](#)



1. Executive Summary & Market Context

Hyperscale infrastructure development is witnessing unprecedented global acceleration. For example, national data centre capacity targets are expanding from 900 MW to 2,700 MW by 2027, drawing over \$15 Billion in capital expenditure. However, EPC (Engineering, Procurement, and Construction) execution models suffer from persistent workflow inefficiencies.

McKinsey reports show that **67%** of large capital projects exceed their initial baseline schedules. The primary contributor to these delays is procurement deviations. When complex equipment (such as medium-voltage switchgear or UPS systems) is ordered, vendor submittals must match structural and electrical requirements. Currently, quality control teams spend **3 to 5 business days** per submittal package manually cross-referencing parameters against specifications. Minor deviations overlooked at this phase create compounding delays when discovered on-site.

AegisEPC introduces an automated project intelligence layer. By extracting, structuring, and verifying technical parameters programmatically using LLMs backed by a deterministic retrieval index, AegisEPC reduces submittal verification times from **days to 8 seconds**, securing compliance at every step.



2. Why Traditional Vector RAG Fails Technical Specs

Most AI document search systems rely on vector databases and embeddings (standard RAG). While suitable for general text queries, vector RAG is fundamentally flawed for engineering specifications due to:

1. CONTEXT SPLITTING & CHUNK TRUNCATION

Vector RAG splits documents into arbitrary token counts (e.g. 500 tokens). If a technical parameter table or clause spans the boundary between Chunk A and Chunk B, the context is broken, leading to incomplete retrieval or total evaluation failure.

2. SEMANTIC DRIFT

Engineering parameters (e.g., "25 kA short-circuit withstand for 3 cycles") share identical semantic profiles with similar parameters ("20 kA short-circuit withstand for 1 cycle"). Vector math cannot resolve these subtle but legally critical variations, causing retrieval of incorrect clauses.

3. LACKING TRACEABILITY

When an auditor needs to reject a vendor submittal, they must reference the exact clause code (e.g., Section 4, Clause 4.3). Vector databases retrieve chunks of raw text but lack the document hierarchy required to output the exact clause path.



3. Proposed Technical Architecture & Ingestion

AegisEPC replaces semantic vector matching with a structured **Clause-Tree Retrieval Index**. The system maps documents into hierarchical trees where every node represents a distinct document section or clause.

Hierarchical Index Structure

```
{
  "sample_specification.md": {
    "title": "Structural Steel & Painting Specification",
    "clauses": [
      {
        "clause": "3.1",
        "section": "3. MATERIAL REQUIREMENTS",
        "title": "Primary Steel Members",
        "text": "Columns, Beams, Girders: ASTM A992/A992M, Min Yield Strength 50 k
      },
      {
        "clause": "4.3",
        "section": "4. TESTING & INSPECTION",
        "title": "Nondestructive Testing",
        "text": "100% Ultrasonic Testing (UT) on full-penetration welds ... "
      }
    ]
  }
}
```

Document Ingestion Pipeline

The backend parses files dynamically. Markdown headings (`#`, `##`, `###`) are read to define section boundaries and clause IDs. Text beneath these headers is captured verbatim. For PDF documents, `pdfplumber` extracts text lines, which are then parsed and structured using header regex rules, converting unstructured layouts into queryable trees.



4. Multi-Agent Design Specifications

Agent 1: RFI & Knowledge Copilot

The RFI Agent resolves technical queries across specification sets. First, it generates a Table of Contents (TOC) for all active specifications or the scoped document. It prompts the LLM to identify the clause target based on this TOC. The backend retrieves the target node's raw text and passes it to the LLM to formulate the final answer, ensuring exact citation formats ('[Source: filename, Clause X.Y]').

Agent 2: Specification & Quality Compliance Agent

The Compliance Agent checks vendor submittals. It runs a **2-step evaluation pipeline**:

1. **Step 1: Parameter Extraction:** The submittal is parsed, and every engineering metric is extracted as a structured object: `{"parameter_name": "...", "value": "...", "unit": "..."}\`.`
 2. **Step 2: Navigation & Cross-Checking Loop:** For each parameter, the agent matches the value against ground-truth spec clauses, calculates compliance, and rates status, severity, and confidence levels.
-

5. Codebase Directory Map & Core Files

```
epc-intelligence/
├── backend/
│   ├── main.py           # FastAPI endpoints, CORS, file upload router
│   ├── doc_parser.py     # Clause-tree structural parser & PDFplumber engine
│   ├── rfi_agent.py      # RFI agent (TOC mapping, query condensation)
│   ├── compliance_agent.py # Parameter extraction & compliance cross-checking
│   └── llm_client.py      # Gemini client wrapper with rate-limiting
└── frontend/
    └── index.html         # Dark-mode dashboard, per-scope chat views
```

Key Implementation Code: `doc\_parser.py`

This code illustrates the indexing and normalized clause retrieval engine:

```
def get_clause_text(source_file: str, clause_number: str) → Optional[dict]:
    # Normalized matching logic handles "7" vs "7.0" vs "07"
    clauses = _clause_trees.get(source_file, [])
    for c in clauses:
        if normalize_clause(c['clause']) == normalize_clause(clause_number):
            return c
    # Fallback to prefix matching
    for c in clauses:
        if normalize_clause(clause_number).startswith(normalize_clause(c['clause'])):
            return c
    return None
```

6. Execution, Verification & Testing Trail

Verification tests validate the performance of the retrieval and evaluation models. Below is the verification run output:

```
=====
```

```
AegisEPC -- Compliance Verification Active
```

```
[OK] Loaded submittal: submittal_vendorA_ups.md
```

```
[Step 1] Extracted 4 parameters from submittal
```

```
[Step 2] Parameter "Battery Autonomy" matches Clause 4.1
```

```
[Verdict] DEVIATION DETECTED: 8 minutes < 10 minutes required (Severity: Critical)
```

```
[Step 2] Parameter "Short-circuit Rating" matches Clause 4.2
```

```
[Verdict] DEVIATION DETECTED: 20 kA < 25 kA required (Severity: Critical)
```

```
[Step 2] Parameter "Nominal Voltage" matches Clause 4.1
```

```
[Verdict] COMPLIANT: 415 V matches 415 V required (Severity: None)
```

```
[OK] Compliance Audit Report generated in 7.4 seconds.
```



7. Business Case & Scalability Roadmap

AegisEPC provides significant returns on investment for EPC project managers:

- **99.8% Time Savings:** Audits that previously took engineers 3–5 days to complete are executed in 8 seconds.
- **Zero Compliance Leakage:** Every parameter is checked with 100% clause traceability, preventing delivery of non-compliant hardware.
- **Minimal Infrastructure Costs:** By parsing documents directly into structured clause indices instead of vector databases, AegisEPC runs efficiently without requiring GPU setups.